

```
from turtle import *
```

```
# 1. 코흐 곡선 재귀 함수
```

```
def koch(length, depth):  
    if depth == 0:  
        forward(length)          # 종료 조건: depth가 0이면 진짜 직선 1개 긋기  
    else:  
        length = length / 3      # 선분 길이를 3분의 1로 축소  
        koch(length, depth - 1)  # 1구간 (평지)  
        left(60)                 # 60도 좌회전 (오르막)  
        koch(length, depth - 1)  # 2구간 (오르막)  
        right(120)               # 120도 우회전 (내리막)  
        koch(length, depth - 1)  # 3구간 (내리막)  
        left(60)                 # 60도 좌회전 (수평 맞추기)  
        koch(length, depth - 1)  # 4구간 (평지)
```

```
# 2. 화면을 지우고 해당 단계(depth)를 새로 그리는 함수
```

```
def draw(depth):  
    tracer(0, 0)                # 화면 깜빡임 없이 즉시 렌더링  
    clear()                     # 이전 단계 그림 깨끗이 지우기  
  
    # 상단 안내 문구 출력  
    penup()  
    goto(0, 230)  
    color("#ffeb3b")            # 노란색 안내 글씨  
    write("[ 키보드 0 ~ 4 키를 눌러보세요 ]", align="center", font=("맑은 고딕", 12, "bold"))  
  
    goto(0, 195)  
    color("#ffffff")            # 흰색 단계 표시  
    count = 3 * (4 ** depth)    # 총 선분 개수 수식:  $3 * 4^{\text{depth}}$   
    write(f"현재 단계 (depth): {depth}단계 | 선분 개수: {count}개", align="center",  
          font=("맑은 고딕", 13, "bold"))  
  
    # 눈송이 시작 위치 잡기 (정삼각형이 화면 중앙에 오도록 이동)  
    goto(-150, 60)  
    setheading(0)               # 오른쪽 방향(0도) 바라보기  
    pendown()  
    color("#00fff5")            # 네온 민트색 눈송이  
    width(1.5)  
  
    # 정삼각형 3개 변에 코흐 곡선 적용
```

```

for _ in range(3):
    koch(300, depth)
    right(120)                # 정삼각형 외각(120도) 회전

update()                     # 완성된 화면을 한 번에 출력

# 3. 화면 및 도화지 기본 설정
setup(650, 680)
title("키보드로 조절하는 코흐 눈송이 시뮬레이터")
bgcolor("#1a1a2e")          # 어두운 남색 배경
hideturtle()                 # 거북이 숨기기

# 4. 키보드 번호(0~4)와 draw 함수 연결
onkey(lambda: draw(0), "0")
onkey(lambda: draw(1), "1")
onkey(lambda: draw(2), "2")
onkey(lambda: draw(3), "3")
onkey(lambda: draw(4), "4")

# 5. 키보드 입력 감지 시작 및 0단계 기본 화면 출력
listen()
draw(0)
done()

```